Studio Nx

JS 295 — 레벨 미터

2026-10-19

JavaScript 295 — 레벨 미터 (`audio/meter.js`, `sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/meter.js`, `studionx/frontend/static/js/pages/sequencer-devices.js` (미터 부분)
작업	S1-11a 재생 오디오 그래프: 흉내 내던 `Math.random()` 미터를 진짜 소리 크기로 교체 (FR-251)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 294 이펙트 체인 관리, 다음 296 화면 연결

1. 하는 일

트랙의 소리 크기(피크)를 좌우로 읽어서 화면 막대의 높이(0에서 100%)로 바꿉니다. 이전에는 재생 중이면 막대를 무작위로 흔들기만 했습니다.

```
트랙의 [pan] 출력 ─▶ ChannelSplitter ─▶ AnalyserNode(왼쪽)  ─▶ 피크 ─▶ dB ─▶ %
                                      └─▶ AnalyserNode(오른쪽) ─▶ 피크 ─▶ dB ─▶ %

화면 타이머 (140ms)  각 막대마다 값을 읽고, 올라갈 때는 바로, 내려갈 때는 9%씩 천천히
```

2. 언제 쓰나

쓰는 곳	하는 일
`MixerGraph.meterFor` (288)	트랙마다 `LevelTap`을 하나 만들어 볼륨과 팬을 거친 소리를 읽습니다. 마스터는 최종 출력을 읽습니다.
`PlaybackSession.meterLevels` (290)	트랙 번호로 `[왼쪽%, 오른쪽%]`를 돌려줍니다.
`sequencer-devices.js`의 `tickMeters`	140ms마다 믹서 줄과 펼침 칸의 막대 `--m` 값을 갱신합니다.
`sequencer-audio.js` (296)	`setMeterSource`로 세션의 `meterLevels`를 화면에 연결하고, 재생 시작 때 `startMeters`로 타이머를 켭니다.

3. 코드

3.1 순수 함수 (`meter.js`)

```
export function peakOf(samples) {
  let peak = 0;
  for (let i = 0; i < samples.length; i++) {
    const v = Math.abs(samples[i]);
    if (v > peak) peak = v;
  }
  return peak;
}
```

```
export function peakToPercent(peak) {
  if (!(peak > 0)) return 0;
  const db = 20 * Math.log10(peak);
  return Math.max(0, Math.min(100, (db - METER_FLOOR_DB) / -METER_FLOOR_DB * 100));
}
```

```
export function meterStep(prev, now, fall) {
  return now >= prev ? now : Math.max(now, prev - fall);
}
```

3.2 소리 크기 읽기 (`meter.js`)

```
read() {
  this.left.getFloatTimeDomainData(this.bufL);
  this.right.getFloatTimeDomainData(this.bufR);
  return [peakToPercent(peakOf(this.bufL)), peakToPercent(peakOf(this.bufR))];
}
```

3.3 화면 갱신 (`sequencer-devices.js`)

```
export function setMeterSource(fn) {
  meterSource = typeof fn === 'function' ? fn : null;
}
```

```
export function startMeters() {
  if (meterTimer) return;
  if (typeof window === 'undefined' || !window.setInterval) return;
  meterTimer = window.setInterval(tickMeters, 140);
}
```

```
function tickMeters() {
  const bars = document.querySelectorAll('.sq-acc.is-live .sq-meter i, .sq-strip.is-live .sq-meter i');
  if (!bars.length) {
    window.clearInterval(meterTimer);
    meterTimer = 0;
    return;
  }
  const playing = isPlaying();
  const cache = {};   // 이번 틱에 같은 트랙을 두 번 읽지 않게
  let busy = false;   // 막대가 하나라도 0 보다 큼
  Array.prototype.forEach.call(bars, function (bar) {
    const prev = parseFloat(bar.style.getPropertyValue('--m')) || 0;
    let now = 0;
    const owner = bar.closest('[data-track-id]');
    if (playing && owner && meterSource) {
      const id = owner.dataset.trackId;
      if (!(id in cache)) cache[id] = meterSource(id);
      const lv = cache[id];
      if (lv) now = lv[bar.matches('i:first-child') ? 0 : 1];
    }
    const next = now >= prev ? now : Math.max(now, prev - 9);   // 올라갈 때는 바로, 내려갈 때는 천천히
    if (next > 0) busy = true;
    bar.style.setProperty('--m', Math.round(next) + '%');
```

```
  });
  // 멈춰 있고 막대가 모두 0 이면 타이머를 끔 (다시 재생하면 startMeters 가 켬)
  if (!playing && !busy) {
    window.clearInterval(meterTimer);
    meterTimer = 0;
  }
}
```

4. 최신 문법으로 쓰면

```
// peakOf 와 peakToPercent 를 최신 문법으로 쓴 모양
export const peakOf = (samples) => samples.reduce((peak, v) => Math.max(peak, Math.abs(v)), 0);

export const peakToPercent = (peak) =>
  peak > 0
    ? Math.min(100, Math.max(0, ((20 * Math.log10(peak) - METER_FLOOR_DB) / -METER_FLOOR_DB) * 100))
    : 0;
```

전통 문법 (실제 코드)	최신 문법
`for` 문으로 최댓값 찾기	`reduce`
`if (!(peak > 0)) return 0;`	삼항 연산자
`function (bar) { ... }`	`(bar) => ...`

5. 왜 이렇게

선택	이유
볼륨과 팬 뒤의 소리를 읽습니다	사용자가 듣는 소리와 같은 크기를 보여야 합니다. 뮤트한 트랙은 막대가 0이 됩니다.
dB 눈금으로 바꿉니다	사람은 소리 크기를 로그로 듣기 때문에, 직선 눈금(0에서 1)으로 그리면 작은 소리가 거의 안 보입니다. -60dB를 0%, 0dB를 100%로 둡니다.
피크를 읽습니다 (RMS가 아님)	클리핑에 가까운지 보는 것이 미터의 주된 쓰임이고, 계산도 단순합니다.
올라갈 때는 바로, 내려갈 때는 천천히	순간적인 큰 소리를 놓치지 않으면서 막대가 덜덜 떨리지 않게 합니다. 한 번에 9%씩 내려갑니다.
같은 틱에 같은 트랙을 한 번만 읽습니다	아코디언과 믹서 줄이 같은 트랙을 함께 보일 때 읽기를 반복하지 않으려는 것입니다 (`cache`).
멈춰 있고 막대가 모두 0이면 타이머를 끕니다	쉬는 동안 140ms마다 도는 일을 없앱니다. 재생을 시작하면 `startMeters`가 다시 켭니다.
막대를 `data-track-id`가 있는 가장 가까운 요소에서 찾습니다	믹서 줄과 펼침 칸 모두 같은 방식으로 트랙을 알아냅니다. 화면 구조가 바뀌어도 이 규칙만 지키면 됩니다.
소리 쪽과 화면 쪽을 `setMeterSource`로 이었습니다	화면 파일이 오디오 세션을 직접 가져오면 서로 가져오는 순환이 생깁니다. 함수 하나만 주고받습니다.

6. 시험

- `docs/handoff/s1-11a_tests/meter_unit.mjs` (14건)는 순수 함수를 확인합니다. 피크는 절댓값의 최댓값이고, 1은 100%, 0은 0%이며, 0.5는 약 90%(-6dB), 0.1은 약 66.7%(-20dB)입니다. 잘못된 값(NaN, 음수)은 0%입니다. 막대는 올라갈 때 바로 오르고 내려갈 때 9%씩 내려갑니다.

- `docs/handoff/s1-11a_tests/ui_meter.mjs` (10건)는 실제 화면에서 확인합니다. 진폭 0.4(-8dB)의 사인파를 재생하면 좌우 막대가 80에서 93% 사이(약 87%)이고, 흉내처럼 흔들리지 않고 변동이 5% 이내입니다. 마스터 막대도 50%를 넘습니다. 정지하면 천천히 내려오고 2초 뒤에 0이 됩니다.
- 시험 중 발견한 일: 처음에는 막대가 계속 0이었습니다. `startMeters`가 펼침 칸을 열 때만 불려서, 믹서 줄의 타이머가 시작된 적이 없었기 때문입니다. 재생을 시작할 때 타이머를 켜고, 모두 0이면 끄도록 고쳤습니다. 이전의 흉내 미터도 같은 이유로 믹서 줄에서는 움직이지 않았습니다.